

E05 — FLOPs Computation: Theoretical Cost Analysis

Overview

This experiment provides a **deterministic theoretical comparison** of the computational cost (in floating-point operations, FLOPs) between Muon and SGD on the matrix sensing problem. Unlike other experiments that involve randomness, this analysis is purely analytical — each algorithm's per-iteration FLOPs count is computed from first principles and scaled by the number of iterations.

Why this matters: Muon requires an SVD of the gradient matrix at each step, which costs $O(d^3)$ FLOPs. SGD does not. Understanding the exact FLOPs penalty is essential for determining whether Muon's iteration-count advantages (observed in E01-E04) translate to actual computational savings.

Experiment ID: E05 | **Problem Domain:** Matrix Sensing | **Algorithms:** SGD vs Muon-Exact

Scientific Question

Hypothesis

The per-iteration FLOPs overhead of Muon's SVD grows as $O(d^3)$, while SGD's per-iteration cost grows as $O(md^2)$. At moderate dimensions with $m = O(dr)$ measurements, the SVD overhead may dominate, making Muon's total FLOPs substantially higher than SGD's despite any iteration-count advantage.

Key Metrics

- **total_flops:** Total floating-point operations for 2000 iterations
- **Per-step FLOPs ratio:** Muon/SGD
- **Scaling with dimension:** How FLOPs grow with d

Theoretical Framework

SGD per-step FLOPs: $3(m + 1)d^2$

Muon per-step FLOPs: $3md^2 + 14d^3$ (includes SVD at $12d^3 + UV^T$ at $2d^3$)

Ratio: $\gamma = 1 + \frac{14d}{3m}$ where $m = 2dr = 10d$ (for $r = 5$), giving $\gamma = 1 + \frac{14d}{30d} = 1.47$ at all dimensions.

Experimental Design

Parameter	Value	Description
d	{50, 100, 200, 500}	Matrix dimensions
r	5	Target rank
m	$2dr$	Measurements (e.g., $m = 500$ for $d = 50$)
iters	2000	Fixed iteration count
Algorithms	SGD, Muon-Exact	Both deterministic cost

Total rows: 2 algorithms x 4 dimensions = 8 rows (deterministic)

Data columns: `d`, `m`, `algo`, `iters`, `total_flops`

The FLOPs are computed analytically using the formulas from the project documentation (Section 4.1.3).

```
In [ ]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
plt.style.use('seaborn-v0_8-whitegrid')

df = pd.read_csv('../results_v3/E05_detailed_results.csv')
print(f"Shape: {df.shape}")
print(f"Columns: {list(df.columns)}")
print(f"Dimensions: {sorted(df['d'].unique())}")
print(f"Algorithms: {df['algo'].unique()}")
print("Data:")
print(df.to_string(index=False))
```

Exploratory Data Analysis

```
In [ ]: # Pivot for comparison
pivot = df.pivot_table(index='d', columns='algo', values='total_flops')
pivot['ratio_Muon_SGD'] = pivot['Muon-Exact'] / pivot['SGD']
pivot['sgd_per_step'] = pivot['SGD'] / 2000
pivot['muon_per_step'] = pivot['Muon-Exact'] / 2000
print("FLOPs comparison (total and per-step):")
print(pivot)

# Theoretical ratio check
print(""
```

```
Theoretical ratio: gamma = 1 + 14d/(3m) = 1 + 14d/(30d) = 1.467")
print("Actual ratios:", pivot['ratio_Muon_SGD'].values)
```

Visualization 1: Total FLOPs by Dimension

```
In [ ]: fig, ax = plt.subplots(figsize=(10, 6))
        dims = sorted(df['d'].unique())
        x = np.arange(len(dims)); width = 0.35

        muon_flops = [df[(df['algo']=='Muon-Exact')&(df['d']==d)]['total_flops'].values[0] for d in dims]
        sgd_flops = [df[(df['algo']=='SGD')&(df['d']==d)]['total_flops'].values[0] for d in dims]

        ax.bar(x - width/2, [f/1e9 for f in muon_flops], width, label='Muon-Exact', color='blue')
        ax.bar(x + width/2, [f/1e9 for f in sgd_flops], width, label='SGD', color='red')

        for i, d in enumerate(dims):
            ratio = muon_flops[i] / sgd_flops[i]
            ax.text(i, max(muon_flops[i], sgd_flops[i])/1e9 * 1.05,
                    f'ratio: {ratio:.2f}', ha='center', fontsize=10, fontweight='bold')

        ax.set_xlabel('Matrix Dimension $d$', fontsize=12)
        ax.set_ylabel('Total FLOPs ($\times 10^9$)', fontsize=12)
        ax.set_title('E05: Total FLOPs for 2000 Iterations - Muon vs SGD', fontsize=14)
        ax.set_xticks(x); ax.set_xticklabels([f'$d$={d}' for d in dims])
        ax.legend(fontsize=11); ax.grid(axis='y', alpha=0.3)
        plt.tight_layout(); plt.show()
```

Visualization 2: FLOPs Ratio (Muon/SGD) vs Dimension

```
In [ ]: fig, ax = plt.subplots(figsize=(10, 6))
        ratios = [muon_flops[i]/sgd_flops[i] for i in range(len(dims))]
        ax.plot(dims, ratios, marker='o', markersize=12, color='#6A4C93', linewidth=2)
        ax.axhline(y=1.0, color='gray', linestyle='--', alpha=0.7, label='Equal cost')

        # Annotate theoretical
        ax.axhline(y=1.467, color='#E63946', linestyle=':', alpha=0.7, label='Theoretical')

        for i, (d, r) in enumerate(zip(dims, ratios)):
            ax.text(d, r + 0.02, f'{r:.2f}', ha='center', fontsize=10, fontweight='bold')

        ax.set_xlabel('Matrix Dimension $d$', fontsize=12)
        ax.set_ylabel('FLOPs Ratio (Muon / SGD)', fontsize=12)
        ax.set_title('E05: FLOPs Ratio vs Dimension', fontsize=14, fontweight='bold')
        ax.legend(); ax.grid(alpha=0.3)
        plt.tight_layout(); plt.show()
```

Visualization 3: Log-Scale FLOPs Comparison

```
In [ ]: fig, ax = plt.subplots(figsize=(10, 6))
for algo, color in [('Muon-Exact', '#2E86AB'), ('SGD', '#F18F01')]:
    flops = [df[(df['algo']==algo)&(df['d']==d)][['total_flops']].values[0] for d in dims]
    ax.plot(dims, flops, marker='s', markersize=10, color=color, label=algo,

ax.set_xlabel('Matrix Dimension $d$', fontsize=12)
ax.set_ylabel('Total FLOPs (log scale)', fontsize=12)
ax.set_title('E05: FLOPs Scaling with Dimension (Log Scale)', fontsize=14, fontweight='bold')
ax.set_yscale('log'); ax.set_xscale('log')
ax.legend(); ax.grid(alpha=0.3)

# Add slope annotations
for algo, color in [('Muon-Exact', '#2E86AB'), ('SGD', '#F18F01')]:
    flops = [df[(df['algo']==algo)&(df['d']==d)][['total_flops']].values[0] for d in dims]
    log_d = np.log(dims)
    log_f = np.log(flops)
    slope = np.polyfit(log_d, log_f, 1)[0]
    ax.text(dims[-1]*0.6, flops[-1]*0.5 if algo=='Muon-Exact' else flops[-1],
            f'{algo}\nslope ~ {slope:.1f}', color=color, fontsize=10, fontweight='bold')

plt.tight_layout(); plt.show()
```

Theoretical Analysis

```
In [ ]: # Verify formulas
print("=" * 70)
print("Theoretical FLOPs Formulas (from documentation Section 4.1)")
print("=" * 70)
print("SGD per step: 3*(m+1)*d^2")
print("Muon per step: 3*m*d^2 + 14*d^3")
print("")
for d in sorted(df['d'].unique()):
    m = int(2 * d * 5) # r=5
    sgd_step = 3 * (m + 1) * d**2
    muon_step = 3 * m * d**2 + 14 * d**3
    sgd_total = sgd_step * 2000
    muon_total = muon_step * 2000
    ratio = muon_step / sgd_step
    print(f"d={d}>4d}, m={m}>6d}: SGD_step={sgd_step:.2e}, Muon_step={muon_step:.2e}, ratio={ratio:.2f}")

# Compare with recorded
rec_sgd = df[(df['algo']=='SGD')&(df['d']==d)][['total_flops']].values[0]
rec_muon = df[(df['algo']=='Muon-Exact')&(df['d']==d)][['total_flops']].values[0]
print(f"Recorded: SGD={rec_sgd:.2e}, Muon={rec_muon:.2e}")
print(f"Match: SGD={np.isclose(sgd_total, rec_sgd)}, Muon={np.isclose(muon_total, rec_muon)}")
print("")
```

Conclusions & Interpretation

Key Findings

1. **Constant FLOPs Ratio:** The per-step FLOPs ratio (Muon/SGD) is approximately **1.47** across all dimensions. This constant ratio arises because:

- $m = 2dr = 10d$ (for $r = 5$)
- Ratio $\gamma = 1 + \frac{14d}{3m} = 1 + \frac{14d}{30d} = 1 + \frac{7}{15} \approx 1.47$
- The d dependence cancels out!

2. **Scaling with Dimension:**

- Both algorithms scale as $O(d^3)$ in total FLOPs for fixed iterations because $m \propto d$.
- On a log-log plot, both show slope ~ 3.0 , confirming cubic scaling.
- Muon's SVD adds a constant 47% overhead regardless of dimension.

3. **Practical Implications:**

- For **wall-clock time** comparisons, the SVD overhead matters less on modern hardware where optimized BLAS routines can achieve near-peak throughput.
- The FLOPs analysis shows that Muon's iteration-count advantage must exceed 47% for total computational cost to be lower than SGD's.
- From E01: Muon needed ~ 10 -20% fewer iterations but 47% more FLOPs per step $\rightarrow$ **Muon is more expensive in total FLOPs.**

4. **Asymptotic Behavior:**

- When $m \gg d$ (many measurements), the ratio approaches 1.0, and Muon becomes computationally competitive.
- When $m \ll d$, the SVD dominates, and Muon's overhead becomes severe.

Summary

The FLOPs analysis reveals that Muon's spectral normalization comes with a **fixed $\sim 47\%$ per-step computational overhead** for the standard $m = 2dr$ measurement regime. Whether this overhead is justified depends on the problem setting: for applications where fast early convergence matters, Muon may still be preferable; for high-precision requirements where many iterations are needed anyway, SGD's lower per-step cost makes it the computationally efficient choice.